

Control Structure, Branching and Looping in Shell Script.

A shell is an environment in which we can run our commands, programs, and shell scripts. The shell provides you with an interface to the UNIX system. It gathers input from you and executes programs based on that input. When a program finishes executing, it displays that program's output.

IF...ELSE STATEMENTS:

If else statements are useful decision making statements which can be used to select an option from a given set of options.

Shell supports following forms of if..else statement,

- IF...FI STATEMENT
- IF...ELSE...FI STATEMENT
- IF...ELIF...ELSE...FI STATEMENT

SYNTAX:

A) Single Decision

```
if <condition>  
then  
### series of code goes here  
fi
```

B) Double Decision:

```
if <condition>  
then  
### series of code if the condition is satisfied  
else  
### series of code if the condition is not satisfied  
fi
```

C) Multiple if condition:

```
if <condition1>  
then  
### series of code for condition1  
elif <condition2>  
then  
### series of code for condition2  
else  
### series of code if the condition is not satisfied  
fi
```

EXAMPLE:

a=10

b=20

if [\$a == \$b]

then

echo "a is equal to b"

else

echo "a is not equal to b"

fi

OUTPUT:

a is not equal to b

CASE...ESAC STATEMENT:

multiple if...elif statements are used to perform a multiway branch. However, this is not always the best solution, especially when all of the branches depend on the value of a single variable.

SYNTAX:

case word in

pattern1)

Statement(s) to be executed if pattern1 matches

;;

pattern2)

Statement(s) to be executed if pattern2 matches

;;

pattern3)

Statement(s) to be executed if pattern3 matches

;;

esac

EXAMPLE:

FRUIT="kiwi"

case "\$FRUIT" in

"apple") echo "Apple pie is quite tasty."

;;

"banana") echo "I like banana nut bread."

;;

"kiwi") echo "New Zealand is famous for kiwi."

;;

esac

OUTPUT:

New Zealand is famous for kiwi.

LOOPS:

Loops are powerful programming tools that enable you to execute a set of command repeatedly. The following types of loops available to shell programmers,+

- **WHILE LOOP**
- **FOR LOOP**
- **UNTIL LOOP**

WHILE LOOP:

The while loop enables you to execute a set of commands repeatedly until some condition occurs. It is usually used when you need to manipulate the value of a variable repeatedly.

SYNTAX:

while command

do

Statement(s) to be executed if command is true

done

Shell command is evaluated. If the resulting value is true, given statement(s) are executed. If command is false then no statement would be not executed and program would jump to the next line after done statement.

EXAMPLE:

a=0

while [\$a -lt 10]

do

echo \$a

a=`expr \$a + 1`

done

OUTPUT:

0

1

2

3

4

5

6

7

8

9

FOR LOOP:

The for loop operate on lists of items. It repeats a set of commands for every item in a list.

SYNTAX:

```
for var in word1 word2 ... wordN  
do  
    Statement(s) to be executed for every word.  
done
```

Here var is the name of a variable and word1 to wordN are sequences of characters separated by spaces (words). Each time the for loop executes, the value of the variable var is set to the next word in the list of words, word1 to wordN.

EXAMPLE:

```
for var in 0 1 2 3 4 5 6 7 8 9  
do  
    echo $var  
done
```

OUTPUT:

```
0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```

UNTIL LOOP:

If the resulting value is false, given statement(s) are executed. If command is true then no statement would be not executed and program would jump to the next line after done statement.

SYNTAX:

```
until command  
do  
    Statement(s) to be executed until command is true  
done
```

EXAMPLE:

a=0

until [! \$a -lt 10]

do

echo \$a

a=`expr \$a + 1`

done

OUTPUT:

0

1

2

3

4

5

6

7

8

9

THE BREAK STATEMENT:

The break statement is used to terminate the execution of the entire loop, after completing the execution of all of the lines of code up to the break statement.

SYNTAX:

break

EXAMPLE:

a=0

while [\$a -lt 10]

do

echo \$a

if [\$a -eq 5]

then

break

fi

a=\$((expr \$a + 1))

done

OUTPUT:

0

1
2
3
4
5